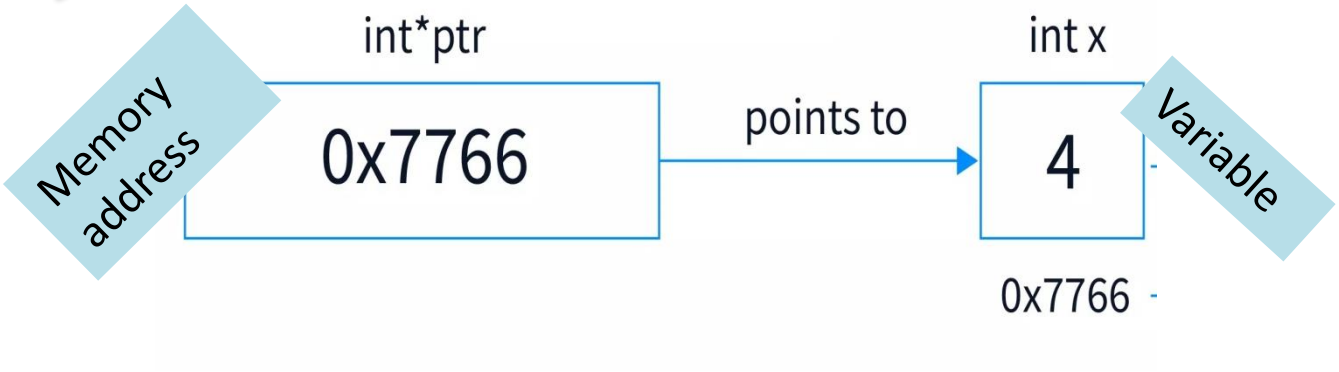
Module 4



Pointers

Part - I

Syllabus

- **Declaration, Operations on pointers**
- **Passing pointer to a function**
- **Accessing array elements using pointers**
- **Processing strings using pointers**
- **Pointer to pointer**
- Array of pointers
- Pointer to function
- Pointer to structure
- Dynamic Memory Allocation

Basics of Pointer

Definition: A pointer is a variable that stores the memory address of another variable

Declaration Syntax: `datatype` ***ptr;***

Example :

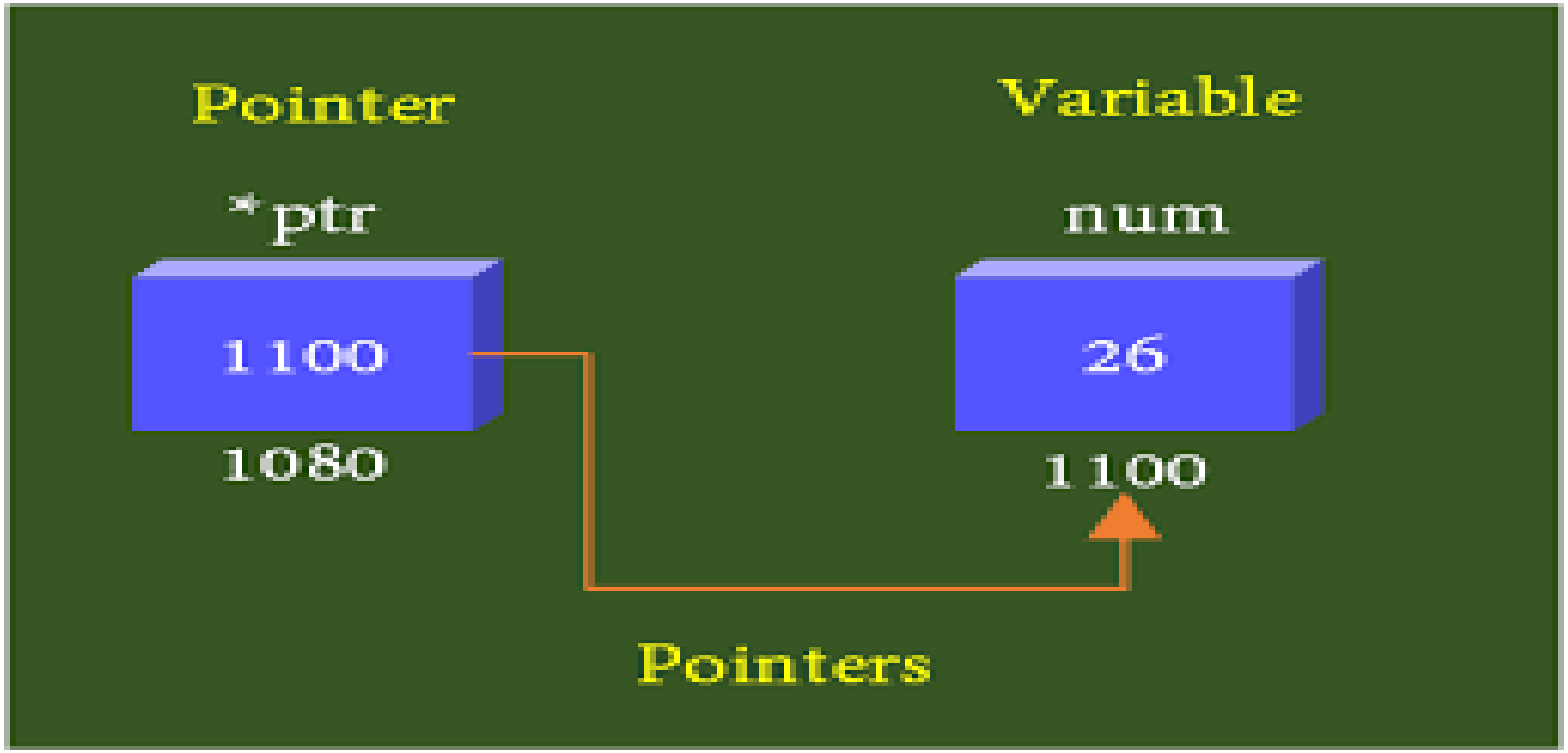
```
int *p1;    //to store the address of int variable
float *p2;  //to store the address of float variable
char *name; //to store the address of char variable
```

Three kinds of pointer declaration

```
int*    ptr
int     *  ptr
int    *ptr
```

Example

```
int   *pa, *pb, * p1;
```



*ptr = 26

ptr = 1100

num = 26

&num = 1100

Pointer = &Variable

Referred to as Pointer to variable (Pointer Variable)

Basics of Pointer

Initialization syntax:

```
datatype *ptr = &var;
```

Example:

```
int x;
int *ptr = &x;
```

Note : This is equivalent to

```
int x, *ptr;
ptr = &x;
```

Basics of Pointer

- `int` pointer can hold the address of `int` variable
- `float` pointer holds the address of a `float` variable.

& : Address operator

*** : Dereference operator (indirection operator)**

The operator & returns the address of the variable associated with it

- **pointer = & variable**
- **variable = * pointer**

Basics of Pointer

- ✓ Pointer is a derived data type
- ✓ It can be used to access and manipulate data stored in memory
- ✓ More efficient in handling arrays and tables
- ✓ Can return **multiple** values from a function
- ✓ Reduce the length and complexity of program
- ✓ Increase execution speed, reduce execution time

Pointer – program to print address

```
#include<stdio.h>
int main()
{
  int x;
  int *ptr1;
  x = 20;
  ptr1 = &x;
  printf("Value of x is : %d\n",x);
  printf("Value of *ptr1 is : %d\n", *ptr1);
  printf("Value of ptr1 is : %p\n", ptr1);
  printf("Value of &x is : %p\n", &x);
  return 0;
}
```

Output:

Value of x is : 20

Value of ***ptr1** is : 20

Value of **ptr1** is : 0x7ffd315ff43c

Value of **&x** is : 0x7ffd315ff43c

Pointer – program_2 to print address

```
#include<stdio.h>
int main()
{
  int x,y;
  int *ptr1,*ptr2;
  x = 20;
  ptr1 = &x;
  y = 30;
  ptr2 = &y;
  printf("Value of *ptr1 is : %d\n", *ptr1);
  printf("Value of ptr1 is : %p\n", ptr1);
  printf("Value of *ptr2 is : %d\n", *ptr2);
  printf("Value of ptr2 is : %p\n", ptr2);
  return 0;
}
```

Output:

Value of *ptr1 is : 20

Address of x → Value of ptr1 is : 0x7ffc29f5a970

Value of *ptr2 is : 30

Address of y → Value of ptr2 is : 0x7ffc29f5a974

Memory Address range

64KB represents 65,536 bytes (8-bit) of memory

65,536 x 8 = 524,288 total bits .

0x0000 to 0xFFFF

0x**** represents hexadecimal equivalent of decimal addresses 0 to 65,535 (Total - 65,536 locations).

Binary	Hexadecimal	Decimal
0000 0000 0000 0000	0000	0
0000 0000 0000 0001	0001	1
⋮ ⋮ ⋮ ⋮	⋮	⋮
1111 1111 1111 1110	FFFE	65534
1111 1111 1111 1111	FFFF	65535

Program_2 - Output

Value of *ptr1 is : 20

Value of ptr1 is : 0x7ffc29f5a970

Value of *ptr2 is : 30

Value of ptr2 is : 0x7ffc29f5a974

Note : The difference is 4.
int x = 20 needs 4 memory location

Program - pointer

```
#include<stdio.h>
int main()
{
   char x = 'R';
   float y = 2.3;
   int z = 6;
   char *ptr1;
   float *ptr2;
   int *ptr3;
   ptr1 = &x;
   ptr2 = &y;
   ptr3 = &z;
```

```
printf(" *ptr1 is: %c\n",*ptr1);
printf(" ptr1 is : %p\n", ptr1);
printf("*ptr2 is: %f\n", *ptr2);
printf(" ptr2 is : %p\n", ptr2);
printf(" *ptr3 is: %d\n",*ptr3);
printf(" ptr3 is : %p\n", ptr3);
return 0;
}
```

Output	
*ptr1 is : R	ptr1 is : 0x7ffec05dd517
*ptr2 is : 2.300000	ptr2 is : 0x7ffec05dd518
*ptr3 is : 6	ptr3 is : 0x7ffec05dd51c

Recollect - **sizeof** - operator

```
#include<stdio.h>
int main()
{
char x = 'R';
float y = 2.3;
int z = 6;
printf("The size of Char is : %ld \n",sizeof(x));
printf("The size of Float is : %ld \n",sizeof(y));
printf("The size of Int is : %ld \n", sizeof(z));
return 0;
}
```

Output: (varies for machines)
The size of Char is : 1
The size of Float is : 4
The size of Int is : 4

Pointer increment and scale factor

```
#include<stdio.h>
int main()
{
int x, *ptr;
ptr = &x;
x = 400;
printf("Value of ptr is :%p\n", ptr);
printf("Value of ptr+1 is:%p\n", ptr+1);
return 0;
}
```

Output:

Value of ptr is : 0x7ffcc0a05d33

Value of **ptr+1** is : 0x7ffcc0a05d37

Pointer increment and scale factor

Scale factor:

When a pointer is incremented, its value is increased by the '*length*' of the data type that it points to. This length is called *scale factor.*

The number of bytes used to store various data types depends on the system.

It can be found by **sizeof** operator.

Programs - Pointer

```
#include<stdio.h>
int main()
{
int *ptr, c;
c = 5;
ptr = &c;
c = 12;
printf("The value of c is : %d\n", c);
printf("The value at address of ptr is: %d",*ptr);
return 0;
}
```

Output:

The value of c is : **12**

The value at address of ptr is : **12**

Programs - Pointer

```
#include<stdio.h>
int main()
{
int *ptr, c;
c = 5;
ptr = &c;
*ptr = 12;
printf("The value of c is : %d\n", c);
printf("The value at address of ptr is : %d", *ptr);
return 0;
}
```

Output:

The value of c is : **12**

The value at address of ptr is : **12**

WAP to find sum of two numbers using pointers

```
#include<stdio.h>
int main()
{
int a, b, *pa,*pb, sum;
pa = &a;
pb = &b;
a = 12; b = 34;
sum = *pa + *pb;
printf("The sum of two numbers is: %d",sum);
return 0;
}
```

Output:
The sum of two numbers is : 46

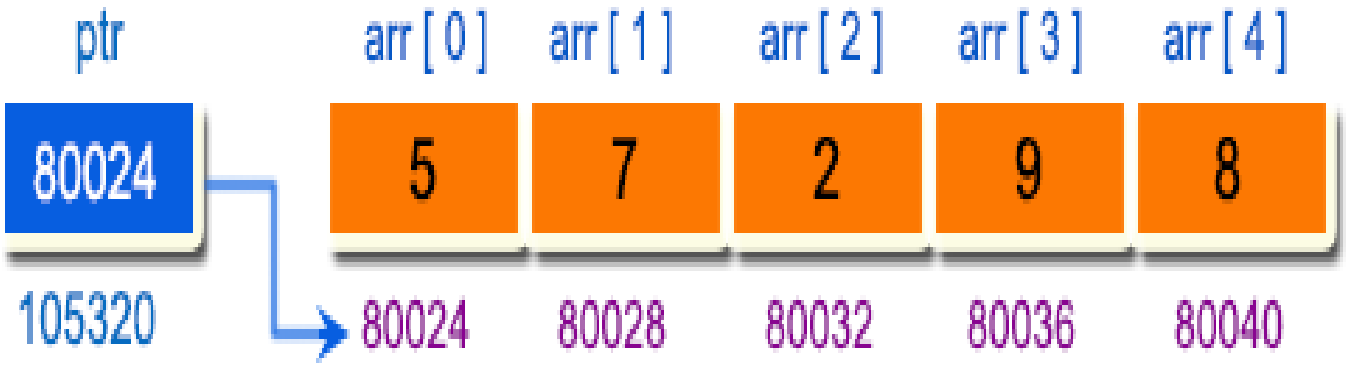
WAP to find the largest of two numbers using pointers

```
#include<stdio.h>
int main()
{
int a,b,*pa,*pb;
pa = &a;
pb = &b;
a = 45; b = 67;
if(*pa > *pb)
   printf("The largest number is : %d", *pa);
else
   printf("The largest number is : %d", *pb);
return 0;
}
```

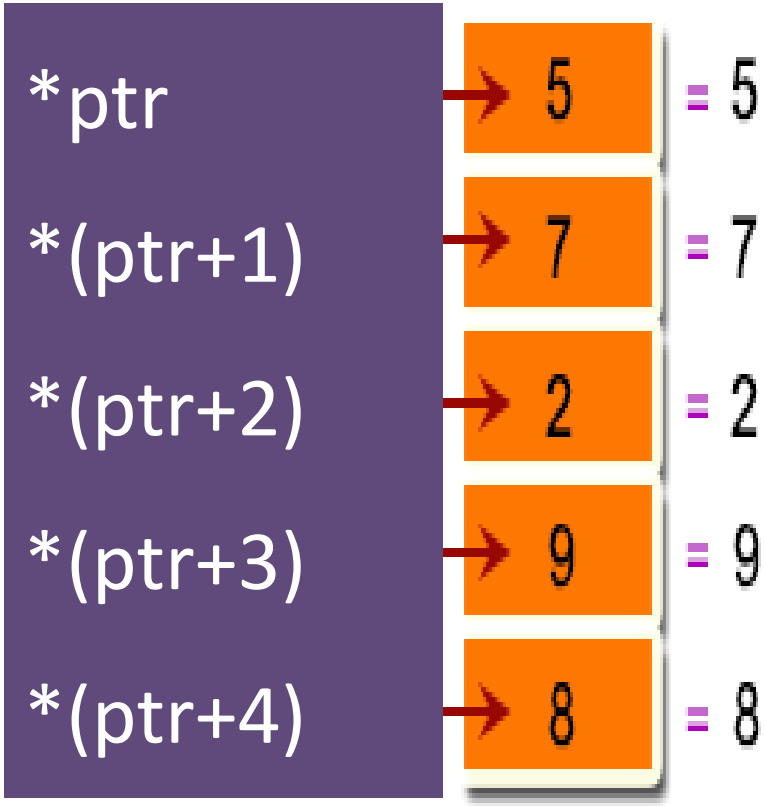
Output:

The largest number is : 67

Accessing array elements using pointers


ptr
arr [0]
arr [1]
arr [2]
arr [3]
arr [4]
80024
5
7
2
9
8
105320
80024
80028
80032
80036
80040

ptr
80024
5
= 80024
ptr + 1
80028
7
= 80028
ptr + 2
80032
2
= 80032
ptr + 3
80036
9
= 80036
ptr + 4
80040
8
= 80040


*ptr
5
= 5
*(ptr+1)
7
= 7
*(ptr+2)
2
= 2
*(ptr+3)
9
= 9
*(ptr+4)
8
= 8

Pointer - arrays

- When an array is declared, the compiler allocates a **base address** and sufficient amount of memory to contain all elements
- The base address is the location of first element – array_name[0]
- Thus for array marks[]

pointer to array is `ptr = marks;`

equivalent to `ptr = &marks[0];`

Relation between ptr and x

Pointer	Address - Array-Index	Address
ptr	&x[0]	1100
ptr + 1	&x[1]	1104
ptr + 2	&x[2]	1108
ptr + 3	&x[3]	110C
ptr + 4	&x[4]	1110

```
scanf("%d", &x[i]);
```
can replaced by

```
scanf("%d", ptr + i); // '&' not needed
```

Pointer	Address	Address - Array Index
p	1100	&x[0]
p + 1	1104	&x[1]
p + 2	1108	&x[2]
p + 3	110C	&x[3]
p + 4	1110	&x[4]

Array Elements
20
30
40
50
60

p	1100
p + 3	110c

*p	20
*p + 3	**23**
*(p+3)	**50**

WAP to input and print the elements of an array using pointers.

```
int x[50],i, N, *ptr;
ptr = x;
printf("Enter the size of the array : ");
scanf("%d", &N);
printf("Enter the elements of the array:\n");
for (i=0; i<N ; i++)
  scanf("%d", ptr + i);  // no need '&'
printf("The given array is : \n");
for (i=0; i<N ; i++)
 {
   printf("%d\n", *(ptr + i));
 }
```

Output

Enter the size of the array : 5 Enter the elements of the array: 14 26 31 44 58	The given array is : 14 26 31 44 58

Base address of array

```
#include<stdio.h>
int main()
{
int x[50];
printf("The address of array begins at:\n");
printf("%p \n %p \n %p", x, &x, &x[0]);
return 0;
}
```

The name of the array can be used as a pointer

Output:

The address of array begins at :
0x7ffd80d3b7c0
0x7ffd80d3b7c0
0x7ffd80d3b7c0

WAP to display the array elements with **array name** as pointer

```
#include <stdio.h>
int main()
{
int x[50], i, N;
printf("Enter the size of array: ");
scanf("%d", &N);
printf("Enter the elements of the array:\n");
for(i = 0; i < N; i++)
     scanf("%d", x+i);
printf("The array elements are :\n");
for(i = 0; i < N; i++)
      printf("%d\n", *(x+i));
return 0;
}
```

Output

Enter the size of the array : 5 Enter the elements of the array: 14 26 31 44 58	The array elements are : 14 26 31 44 58

WAP to find the sum of the elements of an array using pointers.

```
int i, N, x[50], sum, *p;
sum = 0;
p = x;
printf("Enter the size of array : ");
scanf("%d", &N);
printf("The elements are : ");
for(i=0; i < N; i++)
   {
      scanf("%d", p+i);
      sum = sum + *(p+i);
   }
printf("The sum is:%d",sum);
```

Output:

Enter the size of array : 5

The elements are :

10

20

30

40

50

The sum is : 150

WAP to find the smallest element in an array using pointers.

```
#include<stdio.h>
int main()
{
  int i, N, x[50], *p;
  p = x;
  printf("Enter the size of array : ");
  scanf("%d", &N);
  printf("The elements are : ");
  for(i=0; i < N; i++)
  {
    scanf("%d", p+i);
  }
```

Smallest element – pointer- array

```
for(i=1; i<N; i++)
    {
    if(*p > *(p+i))
        {
        *p = *(p+i);
        }
    }
printf("The smallest element is :%d", *p);
return 0;
}
```

Output

Enter the size of array : 4

The elements are :

34

56

0

-9

The smallest element is : -9

Note: The first element *p = x[0] is assumed to be smallest during start; so the “for loop” starts with ‘1’

Processing strings using pointers

Pointer to string

Declaration syntax

```
char  *ptr
```

Initialization syntax

```
char *ptr  = &str
```

or

If declared , initializing can be done as

```
char  str [20], *ptr
ptr = str
```

Pointer to string

```
#include<stdio.h>
int main()
{
char ch,str[50],
*ptr;
int length;
ptr = str;
printf("Enter a word :");
scanf("%s", ptr);
printf("The given word is :
\n");
```

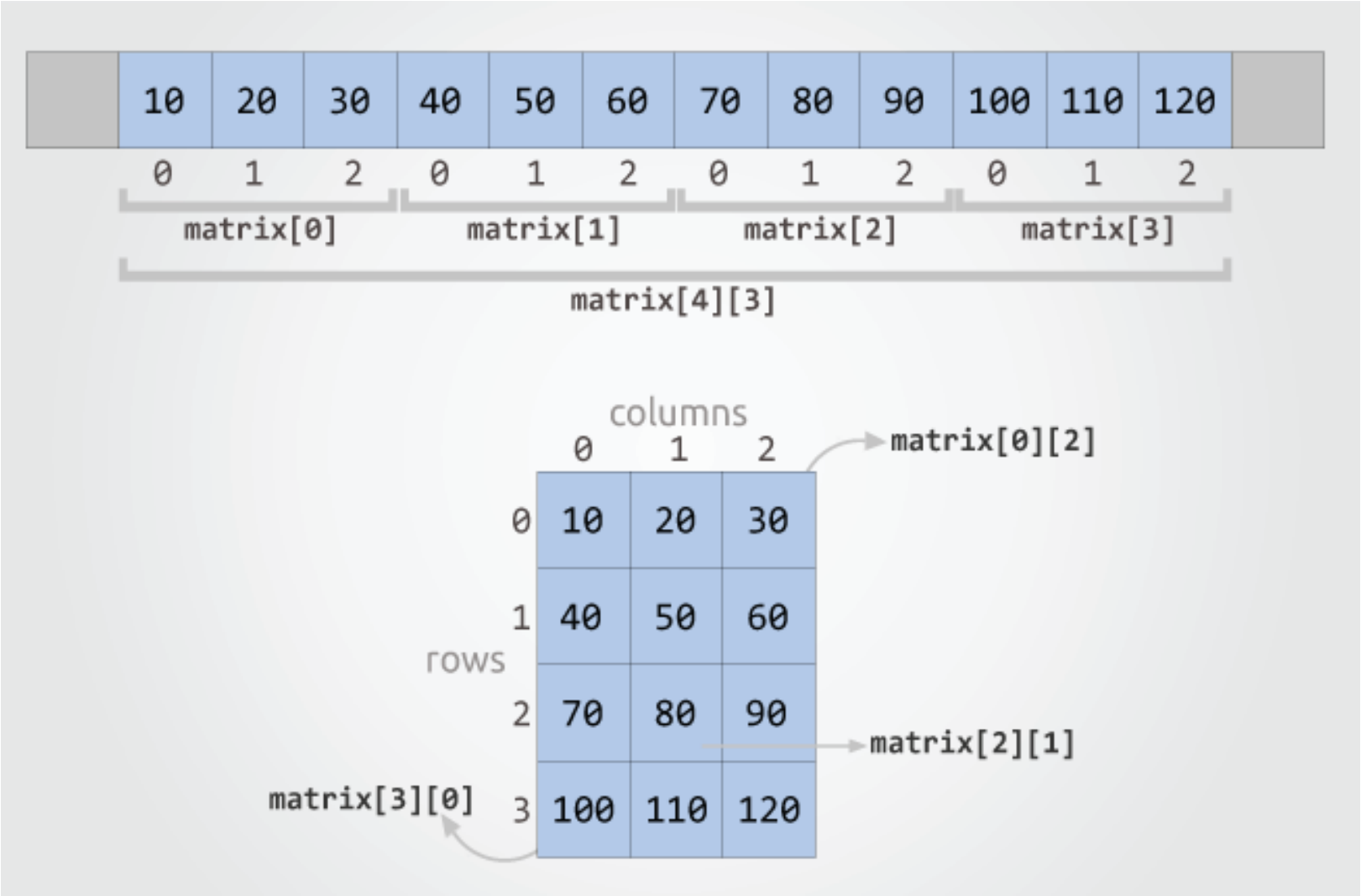
```
 while(*ptr != '\0')
   {
   printf("%c\n", *ptr);
   ptr++;
   }
 length = ptr - str;
 printf("The length of the
   string is : %d", length);
 return 0;
}
```

Output

Enter a word : Lucifer
The given word is :
L
u
c
i
f
e
r
The length of the string is : 7

Pointer to Two dimensional array

Two dimensional array in memory



Memory Map of a 2-Dimensional Array in C

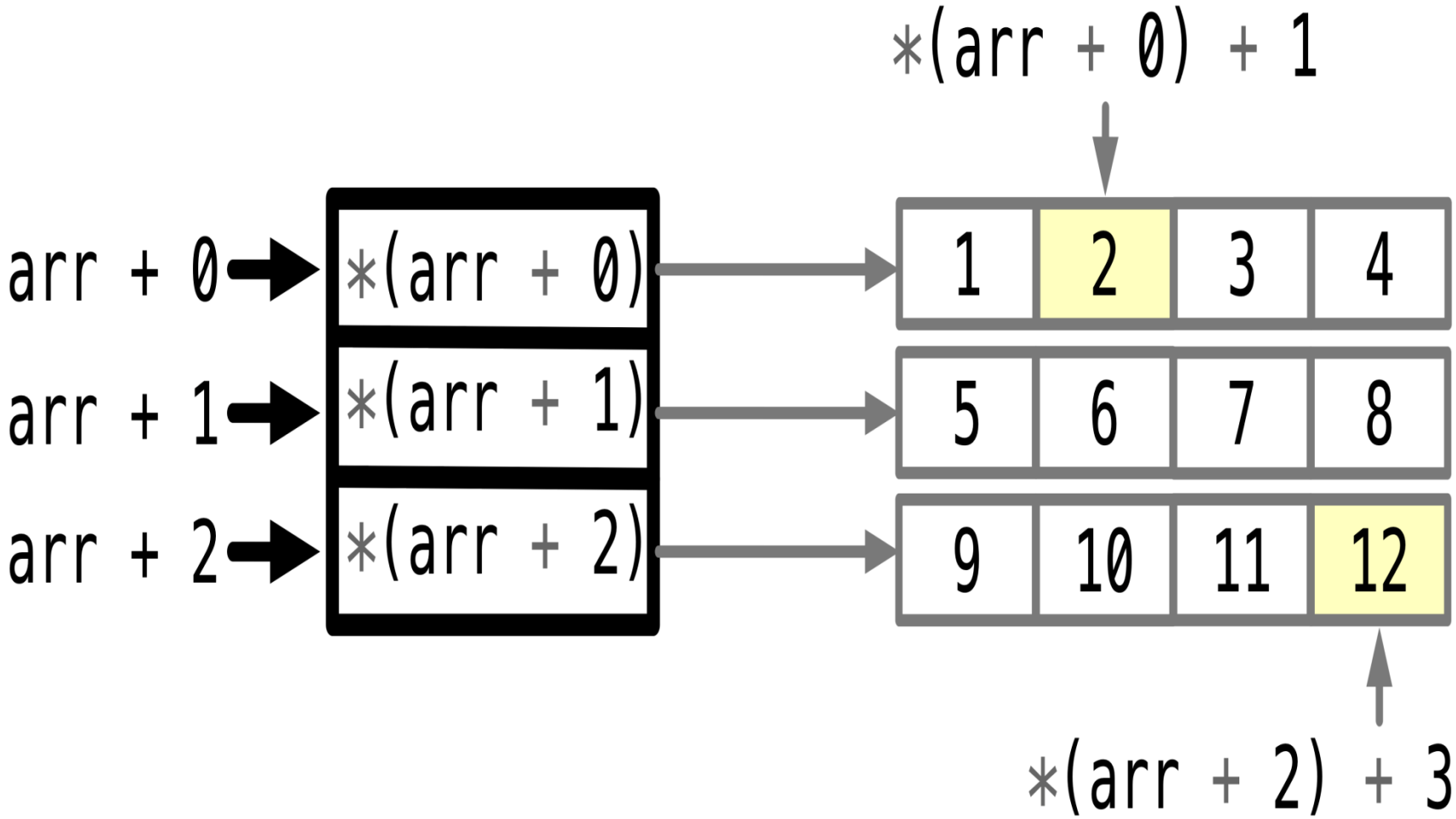
Memory doesn't contain rows and columns.

In memory, the array elements are stored in one continuous chain.

Example – 4X2 matrix

s[0][0]	s[0][1]	s[1][0]	s[1][1]	s[2][0]	s[2][1]	s[3][0]	s[3][1]
1234	56	1212	33	1434	80	1312	78
65508	65510	65512	65514	65516	65518	65520	65522

Dynamic allocation of an array of pointers



Pointer representation for 2-D

	One dimensional array	Two dimensional array
Elements	arr[i]	mat[i][j]
Pointer	arr = ptr	ptr – pointer to first row
	ptr+i	ptr+i - pointer to i^{th} row
		(ptr+i)+j - pointer to i^{th} row, j*th* element

Pointer syntax for 2 - D

Declaration syntax

```
datatype *ptr
```

Initialization syntax

```
datatype *ptr = &arr[0][0]
```

Input and display a **m x n** matrix using pointers

```
#include<stdio.h>
int main()
{
int i, j, m, n, mat[10][20], *ptr;
ptr = &mat[0][0];
printf("Enter number of rows :  ");
scanf("%d", &m);
printf("Enter number of columns : ");
scanf("%d", &n);
```

Input and display a **m x n** matrix using pointers - contd

```
for(i = 0; i < m; i++)
{
 for (j = 0; j < n; j++)
 {
  printf("Enter value of mat[%d][%d]: ",i,j);
  scanf("%d", (ptr+i)+j);
 }
}
```

Input and display a **m x n** matrix using pointers - contd

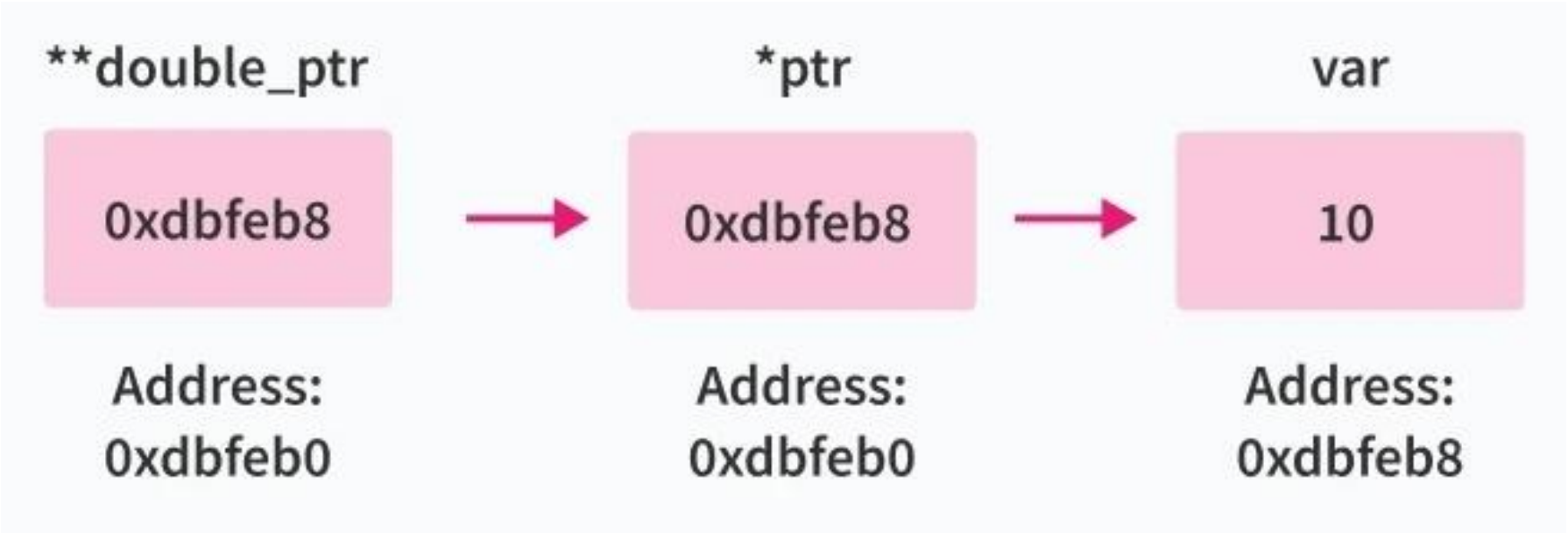
```
printf("The matrix is given below :\n");
for (i = 0; i < m; i++)
  {
  for (j = 0; j < n; j++)
    {
      printf("%d\t", *(ptr+i)+j);
    }
  printf("\n");
  }
return 0;
}
```

Output

```
Enter number of rows : 3
Enter number of columns : 2
Enter value of mat[0][0]: 1
Enter value of mat[0][1]: 2
Enter value of mat[1][0]: 3
Enter value of mat[1][1]: 4
Enter value of mat[2][0]: 5
Enter value of mat[2][1]: 6
```

```
The matrix is given below :
        1   2
        3   4
        5   6
```

Pointer to pointer



Declaration Syntax:
data_type **variable_name

Pointer to pointer

- A pointer variable can store the address of any datatype
  - primary data types - int, float, char,
  - derived data types - arrays, structure

- Pointer can store the address of another pointer too, called "pointer to pointer" (**double pointer**).

WAP using double pointer

```
#include<stdio.h>
int main()
{
 int x, *ptr, **dptr;
 x = 20;
 ptr = &x;
 dptr = &ptr;
 printf("%d\n %p \n %p ", x, ptr, dptr);
 return 0;
}
```

Output:

20
0x7ffde26b38b4
0x7ffde26b38b8

Module 4

Pointers

Part - II

Syllabus

- Declaration, Operations on pointers
- Passing pointer to a function
- Accessing array elements using pointers
- Processing strings using pointers
- Pointer to pointer
- **Array of pointers**
- **Pointer to function**
- **Pointer to structure**
- **Dynamic Memory Allocation**

Array of Pointers

Array of Pointers

An array of pointers stores the addresses of all the elements of the array

Declaration syntax

```
datatype * ptr[Max_size];
```

Initialization syntax

```
char *str_name[ ] = {“Str1",“Str2",“Str3“ };
```

WAP using Array of Pointers

```
#include<stdio.h>
int main()
{
  int a, b, c, i, *p[10];
  a = 6; b = 7; c = 8;
  p[0] = &a;
  p[1] = &b;
  p[2] = &c;
  for (i=0; i<3; i++)
      printf("%d  \t", *p[i]);
  return 0;
}
```

Output:

6 7 8

WAP using Array of Pointers

```
#include<stdio.h>
int main()
{
  int i, N, *ptr[10];
  int var[3] = {20, 30, 40};
  for(i=0; i<3; i++)
      ptr[i] = &var[i];
  printf("The address and variables are :\n");
  for(i=0; i<3; i++)
    printf("%p --- %d  \n", ptr[i],*ptr[i]);
  return 0;
}
```

Output:

The address and variables are :

0x7ffc06d3d554 --- 20

0x7ffc06d3d558 --- 30

0x7ffc06d3d55c --- 40

Array of pointer strings

- An array of string pointers stores the addresses of the strings present in the array.
- Each pointer points to the first character of the string or the base address of the string.

Example

```
char *sports[5] = { "golf", "hockey",
        "football", "cricket", "shooting" };
```

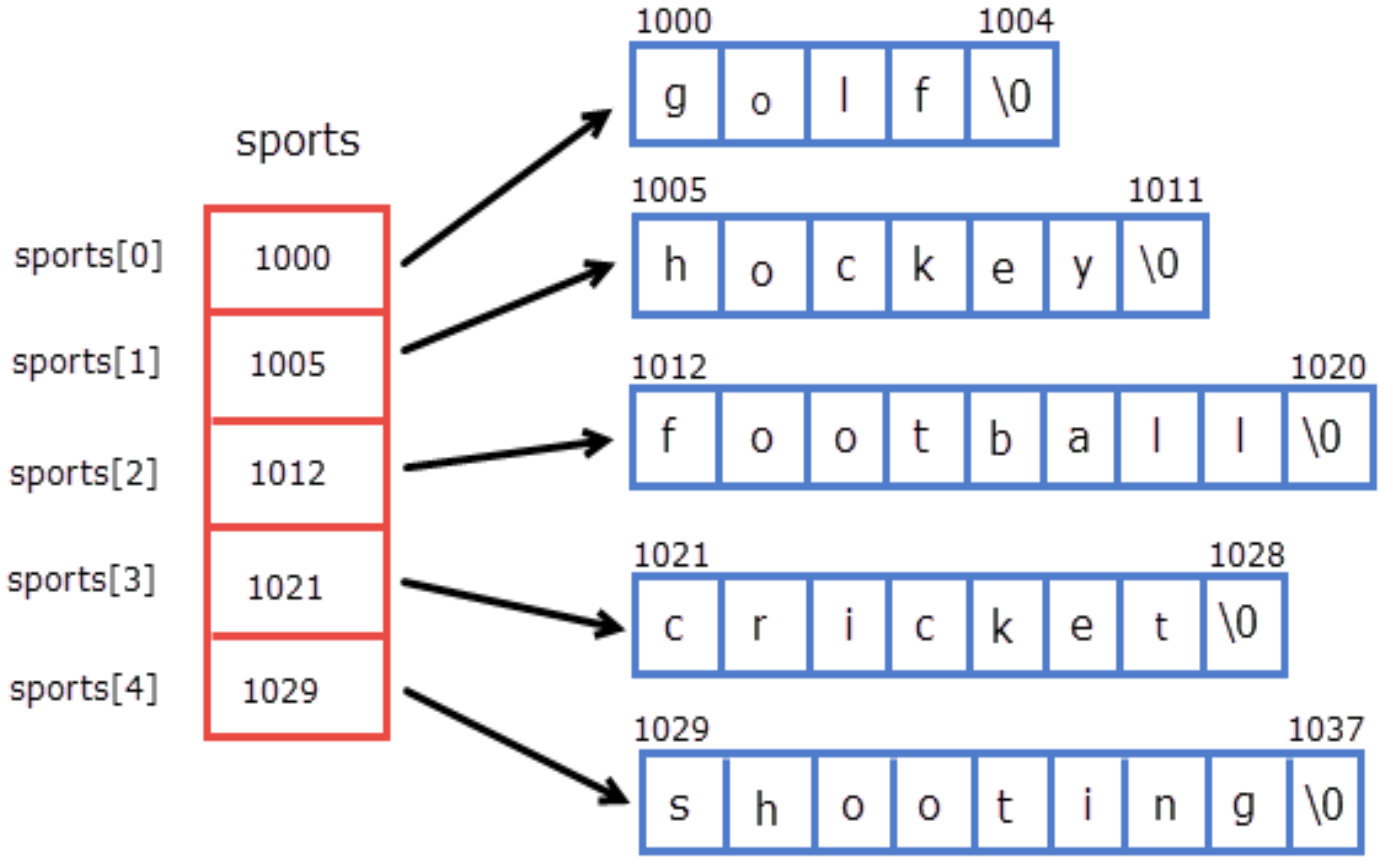
Recollect from Module 2- "2-D strings"

Memory representation of array of strings

```
sports[5][10]
```

1000	g	o	l	f	\0	\0	\0	\0	\0	\0
1010	h	o	c	k	e	y	\0	\0	\0	\0
1020	f	o	o	t	b	a	l	l	\0	\0
1030	c	r	i	c	k	e	t	\0	\0	\0
1040	s	h	o	o	t	i	n	g	\0	\0

Memory representation of array of pointer strings



Inference : Just by creating an array of pointers to string instead of array 2-D array of characters we are saving memory

WAP to demonstrate how to access strings in the array of pointers

```
#include<stdio.h>
int main()
{
int i;
char *name[3] = {"Messi", "Ronaldo", "Neymar"};
printf("The address and names are :\n");
for(i=0; i<3; i++)
  printf(" %p ---- %s  \n", name[i],name[i]);
return 0;
}
```

```
The address and names are :
0x57365e1b0004 ---- Messi
0x57365e1b000a ---- Ronaldo
0x57365e1b0012 ---- Neymar
```

Pointer as function argument

Recollect

Module 3- Functions

- Pass **variables** as arguments to function definition
- Pass **array** as arguments to function definition

Module 4- Pointers

- Pass **pointers** as arguments to function definition

Pointer to function

WAP to find the sum of two numbers using pointers and function

```
void pointer_sum(int *, int *);
int main()
{
  int a, b, *pa, *pb;
  a = 5; b = 4;
  pa = &a; pb = &b;
  pointer_sum(pa, pb); // pointer_sum(&a, &b);
  return 0;
}

void pointer_sum(int *pa, int *pb)
{
  int sum;
  sum = *pa +*pb;
  printf("The sum of two numbers is : %d\n", sum);
}
```

Address of variable passed as arguments

Output:
The sum of two numbers is : 9

Sum of elements of array using pointer and user defined function

```
#include<stdio.h>
int sum_array(int N, int *ptr);
int main()
{
int N, sum, arr[100], *ptr;
ptr = arr;
printf("Enter the size of array : ");
scanf("%d", &N);
sum = sum_array(N, ptr);
printf("The sum of array elements is : %d", sum);
return 0;
}
```

Address of array passed as arguments

Sum of elements of array using pointer and user defined function

```
int sum_array(int N, int *ptr)
{
int i, arr[100], sum =0;
printf("Enter the elements of array :\n");
for(i=0;i<N;i++)
 {
  scanf("%d", ptr +i);
  sum = sum + *(ptr+i);
 }
 return(sum);
 }
```

Output:

Enter the size of array : 5

Enter the elements of array :

3 5 8 9 6

The sum of array elements is : 31

Pass by value (Call by value)
Module - 3

Pass by reference (Call by reference)
Module - 4

Pass by value

- When arguments are passed by value then the **copy** of the actual parameters is transferred from calling function to the called function definition in formal parameters.
- Now any changes made in the formal parameters in called function definition will **not** be reflected in actual parameters of calling function.
- In call by value, **different memory** is allocated for actual and formal parameters

Program – pass by value

```
void increment(int);
int main()
{
int x = 10;
printf("Before function call x is : %d\n", x);
increment(x);          // Variable passed as argument
printf("After function call x is : %d\n", x);
return 0;
}
void increment(int x)
{
x = x+1;
x++;
printf("  The value of x within function is: %d\n\n", x);
}
```

Output:

Before function call x is : **10**

The value of x within function is : 12

After function call x is : **10**

Program – pass by value (Swapping)

```
void swap_value(int, int);
void main()
{
int a = 10, b = 25;
printf("Before calling swap function a is : %d, \t b is : %d\n",a,b);
swap_value(a,b);      // Variable passed as argument
printf("After calling swap function a is : %d, \t b is : %d\n",a,b);
}

void swap_value(int a,int b)
{
int temp;
temp = a; a = b; b = temp;
printf("The swapped values in fn defn a is :%d, \t b is : %d\n",a,b);
}
```

Output- pass by value (swapping)

Before calling swap function **a is : 10, b is : 25**

The swapped values in fn defn **a is :25, b is : 10**

After calling swap function **a is : 10, b is : 25**

Pass by reference

- When arguments are passed by reference then the **address** of the actual parameters is transferred from calling function to the called function definition in formal parameters.
- Now any changes made in the formal parameters in called function definition will be reflected in actual parameters of calling function.
- In call by reference, the **memory allocation is similar** for both formal parameters and actual parameters.

Pass by reference

```
void increment(int *);
int main()
{
int x , *px;
x = 10;
px = &x;
printf("Before function call x is : %d\n", x);
increment(px);     // Address (Pointer) passed as argument
printf("After function call x is : %d\n", x);
return 0;
}
void increment(int *px)
{
*px = (*px)+1;
(*px)++;
printf("The value of *px within function is : %d\n", *px);
}
```

Output – Pass by reference

Before function call x is : 10

The value of *px within function is : **12**

After function call x is : **12**

Pass by reference

```
void swap_ref(int *, int *);
void main()
{
int a = 10, b = 25;
printf("Before calling swap function a is : %d , b is : %d\n",a,b);
swap_ref(&a,&b); // Address (Pointer) passed as argument
printf("After calling swap function a is : %d , b is : %d\n",a,b);
}

void swap_ref(int *pa,int *pb)
{
int temp;
temp = *pa; *pa = *pb; *pb = temp;
printf("The swapped values in FnDefn a is:%d ,b is : %d\n", *pa,*pb);
}
```

Output – Pass by reference (Swapping)

Before calling swap function a is : 10, b is : 25

The swapped values in FnDefn a is :25, b is : 10

After calling swap function a is : 25, b is : 10

Multiple return values using pointer

We can return more than one values from a function by using the method called

1) Call by address
2) Call by reference
3) Pass by reference (address)

Use pointer and user defined function to find the quotient and remainder

```
#include<stdio.h>
void div(int a, int b, int *quotient, int *remainder)
{
   *quotient = a / b;
   *remainder = a % b;
}
int main()
{
int a, b, q, r;
printf("Enter the numbers : ");
scanf("%d %d", &a, &b);
div(a, b, &q, &r);
printf("Quotient is: %d\n Remainder is: %d\n", q, r);
return 0;
}
```

Output:
Enter the numbers :
7 2
Quotient is: 3
Remainder is: 1

Note: Two values - Quotient and remainder are returned to main program

Use pointer and user defined function to find the area and perimeter

```
#include<stdio.h>
#define PI 3.14
void perimeter_area(float, float*, float*);
int main()
{
float r, area, perimeter;
printf("Enter the radius : ");
scanf("%f", &r);
perimeter_area(r, &area, &perimeter);
printf("The area of the circle is : %f\n", area);
printf("The perimeter of the circle is : %f", perimeter);
return 0;
}
```

Use pointer and user defined function to find the area and perimeter -contd

```
void perimeter_area(float r, float *area, float *perimeter)
  {
  *area = PI*r*r;
  *perimeter =  2*PI*r;
  }
```

Output:

Enter the radius : 10

The area of the circle is 314.000000

The perimeter of the circle is 62.799999

POINTER TO STRUCTURE

Pointer to structure

It is pointer variable that points to the address of a structure

Syntax

```
struct keyword Name_of_struct  *ptr_name
```

Initialization of structure pointer

```
struct Student
{
int id;
char name[30];
int age;
char gender[15];
};
struct Student student1;
struct Student *ptr = &student1;
```

Pointer to structure

It is pointer variable that points to the address of a structure

Example

```
struct inventory
{
    char name[30]
    int number;
    float price;
} product[2], *ptr
```

```
ptr  = product
```

assigns zeroth element of the product to ptr

Accessing Pointers to Structures

To access members of the structure using a pointer, we have two methods :

- Using Indirection Operator(*) and Dot (.) operator
- Using Arrow Operator (->)

Accessing Pointers to Structures

```
#include<stdio.h>
#include<string.h>
int main()
{
struct Employee
{
    int id;
    char name[100];
    int salary;
};

struct Employee employee1;
struct Employee *ptr;
```

Accessing Pointers to Structures

```
ptr = &employee1;
employee1.id = 1;
employee1.salary = 10000;
strcpy(employee1.name , "Susan");
printf("Name is : %s\n",(*ptr).name);
printf("The ID is : %d\n", (*ptr).id);
printf("Salary is: %d", (*ptr).salary);
return 0;
}
```

Using Indirection Operator *, .

Accessing Pointers to Structures

```
printf("Name is : %s\n",ptr ->name);
printf("The ID is : %d\n", ptr-> id);
printf("Salary is: %d", ptr ->salary);
```

Using Arrow operator

Output:
Name is : Susan
The ID is : 1
Salary is: 10000

Dynamic memory allocation

Dynamic memory allocation

- It is **allocating memory** while the **program is running**, instead of at the time of writing the code.
- It allows to request memory from the system as needed using functions like malloc(), calloc(), or realloc(), and release it using free().
- This gives flexibility to handle data whose size may not be known in advance, like user input or dynamic data structures such as linked lists.

Dynamic memory allocation

Function	Purpose	Initializes Memory?	Can Resize?	Requires free()?	Header File
`malloc()`	Allocates single block of memory	No	No	Yes	`<stdlib.h>`
`calloc()`	Allocates multiple blocks & zeroes them	Yes	No	Yes	`<stdlib.h>`
`realloc()`	Resizes previously allocated memory	No (old data kept)	Yes	Yes	`<stdlib.h>`
`free()`	Deallocates previously allocated memory	N/A	N/A	Itself releases	`<stdlib.h>`

malloc()

- The malloc() function stands for Memory Allocation.
- It is used to dynamically allocate a single block of memory of the specified size during the execution of the program.

calloc()

- The calloc() function stands for Contiguous Allocation.
- It is used to allocate memory for multiple elements and also initializes all of them to zero.

realloc()

- The realloc() function stands for Reallocation of Memory.
- It is used to resize an already allocated memory block during program execution—either to expand or shrink the existing block.

free()

- The free() function is used to release dynamically allocated memory back to the system.
- It helps prevent memory leaks and ensures efficient memory usage during the program's lifecycle.